

TradeFlow

Software Architecture Document

Version 1.0

Date: June 2026

Author: Christopher Simonson

Table of Contents

1. Introduction
2. Architectural Representation
 - 2.1 Architecture Overview
3. Architectural Goals and Constraints
4. Use Case View
5. Logical View
6. Process View
7. Deployment View
8. Security Architecture
9. Performance and Scalability
10. Reliability and Recovery
11. Technology Stack
12. Architectural Decisions
13. Future Considerations

1. Introduction

1.1 Purpose

This document describes the architecture of TradeFlow: its responsibilities, its internal boundaries, the runtime workflows that move an alert from receipt to execution, and the external systems it depends on. It is intended for engineers who will operate or extend the system and for stakeholders who require an accurate account of how it behaves. It records the architecture as built rather than proposing a target state.

1.2 Scope

TradeFlow ingests trading alerts from the Xtrades platform, evaluates each alert against a configurable risk policy, sizes and routes approved entries to Interactive Brokers for execution, manages the resulting positions and their broker-side exit orders, records trade outcomes for later analysis, and exposes a read interface for querying ingested alerts. At the time of writing the system trades against an Interactive Brokers paper account.

The description that follows reflects the system as implemented. Where behaviour is referenced in configuration or planning but is not yet implemented, it is identified as a future consideration rather than current functionality.

1.3 Definitions and Acronyms

Term	Definition
Alert	A signal from Xtrades describing a trade action by a tracked trader.
Entry / Exit	Alerts that open a position (BTO, AVG) or close one (STC, BTC).
IBKR	Interactive Brokers, the executing broker.
TWS API	The Interactive Brokers socket API, accessed through IB Gateway.
OCA	One-Cancels-All order group. The fill of one order cancels the others.
ODTE	An options contract expiring on the current trading day.
Risk tier	The classification (standard, high, lotto) that governs sizing and stop behaviour.
Market regime	The day's assessment of trading conditions, expressed as a chop score.
ET	United States Eastern Time, the reference zone for market hours.

1.4 References

1. Interactive Brokers TWS API documentation.
2. Xtrades alert API and SignalR notification hub.
3. PostgreSQL and the Npgsql Entity Framework Core provider.
4. .NET 10 hosting and configuration documentation.

2. Architectural Representation

The architecture is presented through four views. The Use Case View states the events the system responds to and the actors involved. The Logical View describes the components, their responsibilities, and the dependencies between them. The Process View describes the concurrent runtime workflows and how an alert progresses through the system. The Deployment View describes the runtime environment and the external systems the application integrates with.

2.1 Architecture Overview

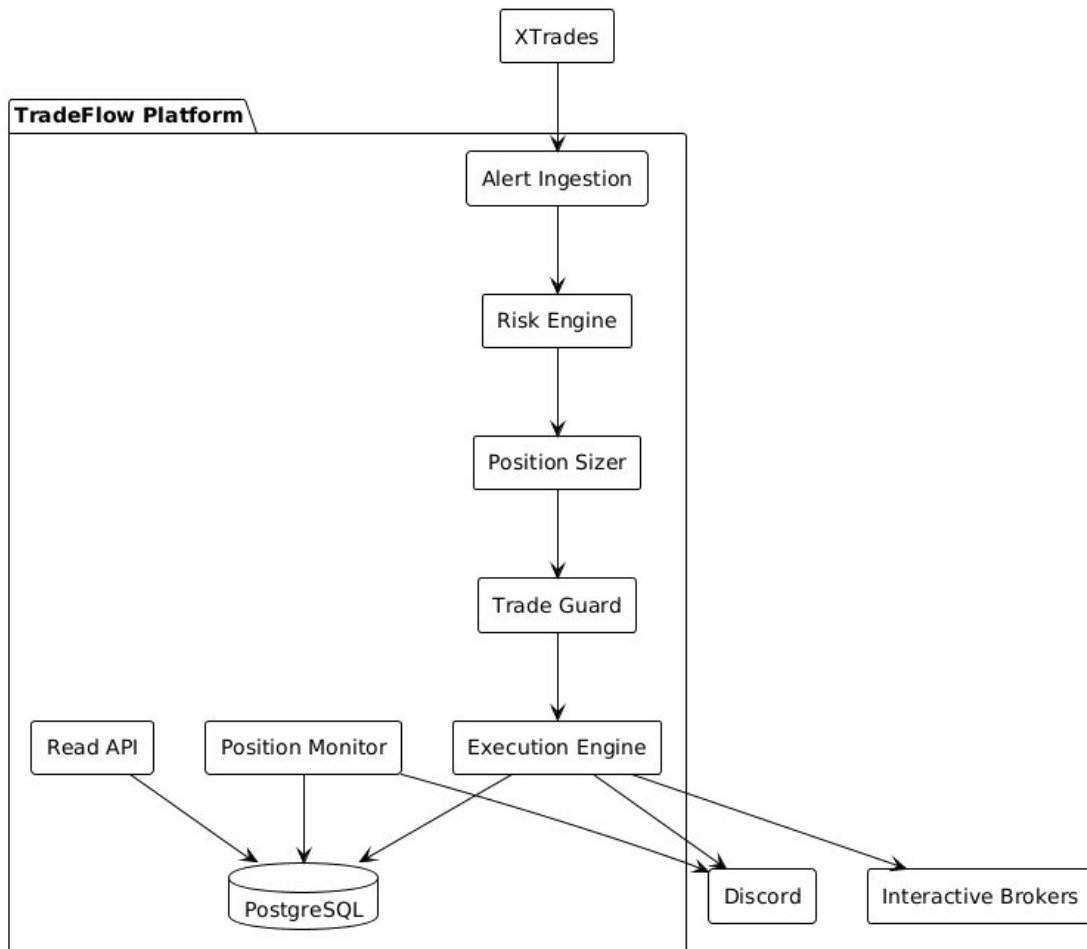
TradeFlow is an event-driven trading automation platform. It listens for trade alerts published by tracked traders on the Xtrades platform, decides whether each alert should be acted on, and, when it should, places and manages the corresponding order with Interactive Brokers. The platform runs unattended through the trading day and records every decision and outcome so that its behaviour can be reviewed afterwards.

The system is organized around a single execution core that owns the full path from alert to managed position. Alerts arrive through two paths, a real-time feed for low latency and a polling interface for reliability, and both feed one processing pipeline so that behaviour is identical regardless of source. Each alert is normalized, classified, and evaluated against a configurable risk policy. Approved entries are sized into orders, screened by an account-level safety control, and routed to the broker through a single abstraction that can run in simulation, paper, or live mode. Positions are then protected by broker-side exit orders and tracked until they close.

Risk management is not a single component but a layered concern that runs through the architecture. A risk engine acts as an admission gate that rejects ineligible alerts before any order is sized. A trade guard then applies account-level limits as the last control before capital is committed. A daily market-regime assessment can tighten both automatically when conditions are unfavourable. The choice between simulated and live execution defaults to simulation, so the system cannot place real orders unless explicitly configured to.

The architecture is organized this way to keep three concerns separate and independently verifiable: the policy that decides whether to trade, the mechanics of placing and protecting an order, and the broker that executes it. Isolating the broker behind an abstraction lets the same workflow be tested without a live connection and run identically in paper and live trading. Persisting open positions as the source of truth lets the system recover its state after a restart. Splitting ingestion into two paths balances latency against reliability without duplicating the logic that follows.

TradeFlow - Executive Architecture Overview



3. Architectural Goals and Constraints

The architecture is shaped by the following goals and constraints, each of which is reflected in the structure of the system.

Dual-path ingestion. Alerts must be received both by polling the Xtrades REST API and through the Xtrades real-time feed. The two paths exist to balance latency against reliability: the real-time feed minimizes the delay between a trader's action and execution, while polling provides a dependable backstop when the feed is unavailable. Both paths must yield identical results, and the same alert must never be processed twice.

Risk evaluation as an admission gate. No order may reach the broker without passing an ordered risk policy. Both approved and rejected alerts must be retained so that risk decisions can be reviewed after the fact.

Broker independence. Execution must be switchable between a simulation broker and the live broker without code change, defaulting to the simulation broker so that an unconfigured environment cannot place real orders.

State durability. The system must survive process restarts without losing awareness of open positions or their resting exit orders.

Externalized secrets. All credentials and endpoints must be supplied at runtime and must never be committed to source control or embedded in deployable images.

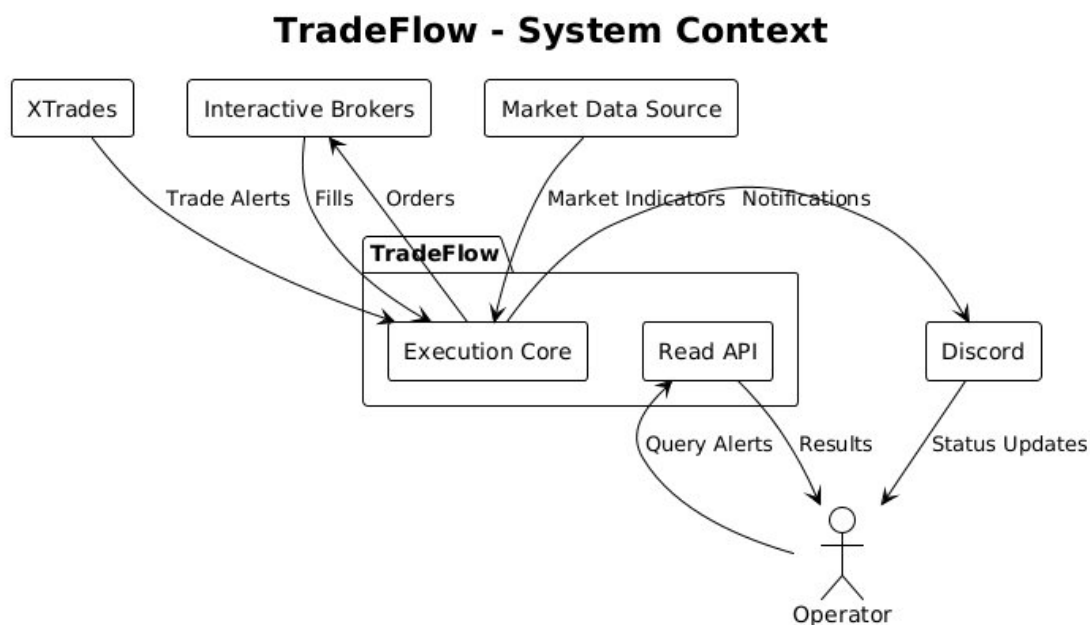
Single broker session. The broker client is single-threaded by contract. All broker access must pass through one connection, which bounds order throughput to a single session.

Calendar and time-zone discipline. Trading and scheduled activity are constrained to United States market hours and market days and are evaluated in Eastern Time regardless of where the host runs.

Unattended operation. The system runs for extended periods without an operator present and must recover from transient network faults, broker disconnections, and malformed market data without manual intervention.

4. Use Case View

TradeFlow responds to alerts produced by tracked traders and to scheduled time-of-day events. The external actors are the Xtrades alert source, the Interactive Brokers execution venue, the Discord notification channels, and a consumer of the read interface.



4.1 Architecturally Significant Use Cases

Process an entry alert. An entry alert is normalized, classified, and evaluated against the risk policy. If approved, it is sized into an order, admitted by the trade guard, and routed to the broker. The decision and the outcome are recorded and announced.

Process an exit alert. An exit alert is matched against the positions the system holds and, if a match exists, closes the position. Exits are honoured regardless of the risk decision, because

the obligation to close an open position is independent of whether the system would have opened it.

Handle a broker-side exit. When the broker fills a trailing stop or target that the system placed, it notifies the system directly. The position is reconciled and closed without the system polling for price.

Run scheduled market-day tasks. At fixed points on market days the system establishes the day's market regime, reports health and position summaries, and performs end-of-day handling.

Query ingested alerts. A consumer retrieves ingested alerts, with filtering and pagination, or a single alert by identifier.

5. Logical View

5.1 Architecture Overview

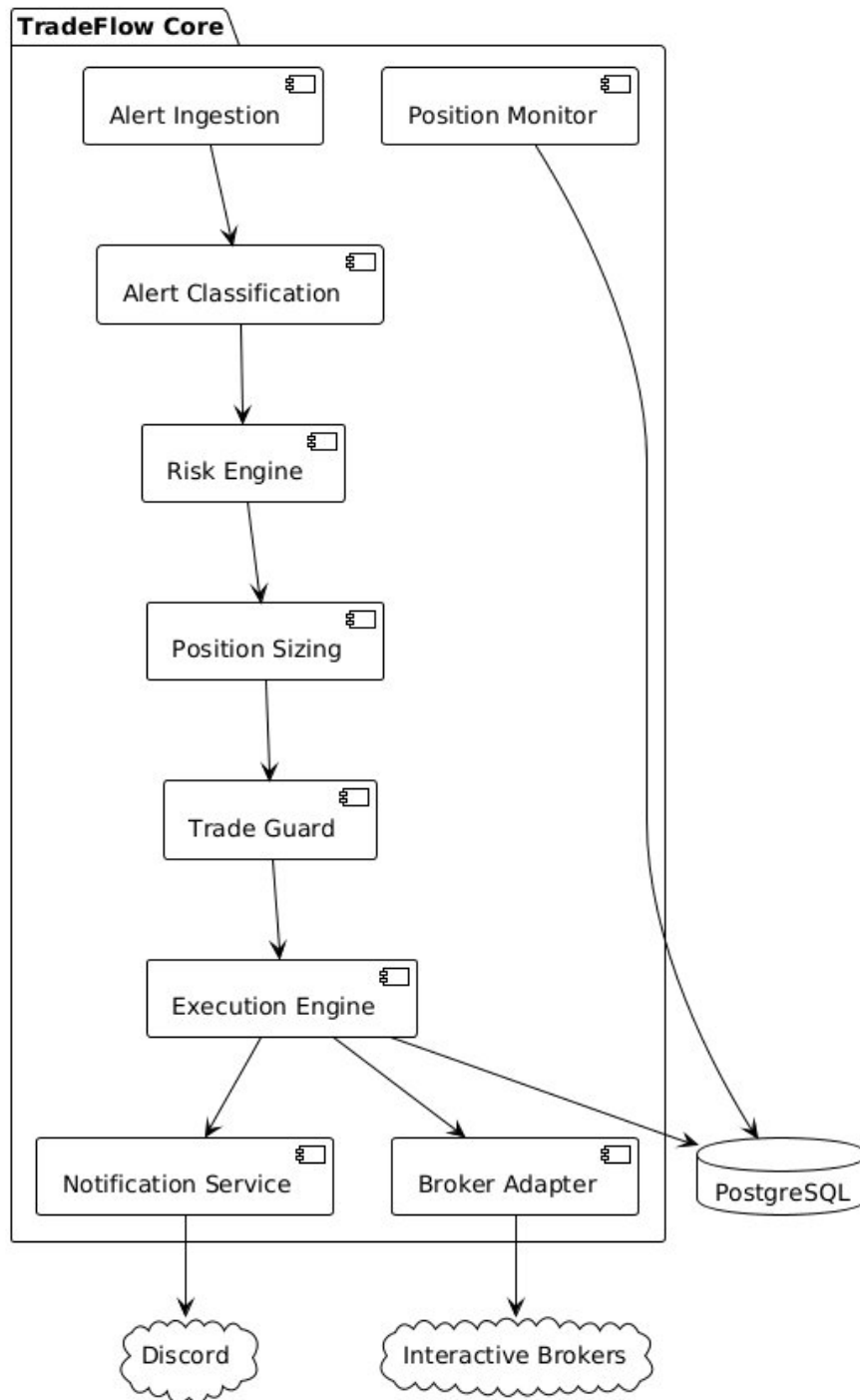
The system is organized into a shared domain library, an execution core, a read interface, a reporting tool, and a test suite.

Component	Responsibility
Domain and risk library	Owns the alert model, the alert classifier, and the risk engine with its rules. Shared by the core and the tests.
Execution core	The operational center of the system. Owns ingestion, risk evaluation, sizing, execution, position management, scheduling, and persistence.
Read interface	Serves a read-only view of ingested alerts over HTTP.
Reporting tool	Produces performance reports from recorded trade outcomes, run as a separate process.
Test suite	Unit coverage of the risk engine, sizing, and execution, and integration coverage against a real database and the API host.

The dependency direction is deliberate. The execution core and the read interface both depend on the domain library and on the shared persistence types; nothing depends on the core's execution logic except the tests. This keeps the risk policy and the alert model free of execution and infrastructure concerns and allows the read interface to reuse the persistence model without inheriting the trading runtime.

5.2 Major Components

TradeFlow - Component Architecture



Ingestion. Two services receive alerts, one by polling and one from the real-time feed. Each owns only the concern of acquiring alerts and handing them to a shared processing path; neither contains risk or execution logic. This separation is what allows the two paths to remain behaviourally identical.

Risk engine. The risk engine evaluates an alert against an ordered set of rules and returns the first failure or an approval. It owns the admission policy and nothing else. Rules cover entry-only filtering, trader approval and quality scoring, blocked symbols, instrument-price floors, and the suppression of high-risk and lotto entries. The engine is assembled from configuration at startup, so policy changes do not require code changes.

Position sizing. The sizer converts an approved alert into an order, deciding quantity, stop, target, and trailing behaviour from the instrument type and risk tier. It owns the translation from a qualitative signal to a quantified order. Options risk tiers are derived from time to expiry rather than taken at face value from the source, which lets the system apply tighter controls to short-dated contracts.

Trade guard. The guard is the final admission control before capital is committed. It owns the account-level safety policy: it prevents duplicated exposure on a single underlying, enforces position and exposure limits, ensures orders never exceed available capital, and halts new entries once a daily loss limit is reached. It also holds the in-memory record of open positions used for exit matching.

Execution. Execution is the single workflow shared by both ingestion paths. It coordinates sizing, admission, order placement, exit-order protection, recording, and notification, and is the only path through which orders reach the broker.

Broker integration. A single broker abstraction decouples execution from the broker implementation. Two implementations exist: a simulation broker that returns modelled fills and the Interactive Brokers implementation. The active implementation is chosen at startup, defaulting to simulation. All broker interaction is serialized through one connection because the broker client is single-threaded by contract.

Notifications. The notification component publishes events to separate Discord channels for approved signals, confirmed executions, operational health, daily summaries, and critical infrastructure events. Notification failures are isolated and never block trade processing.

Persistence. Three stores are maintained: ingested alerts with their risk decisions, recorded trade outcomes for analysis, and open positions as the durable record of live exposure.

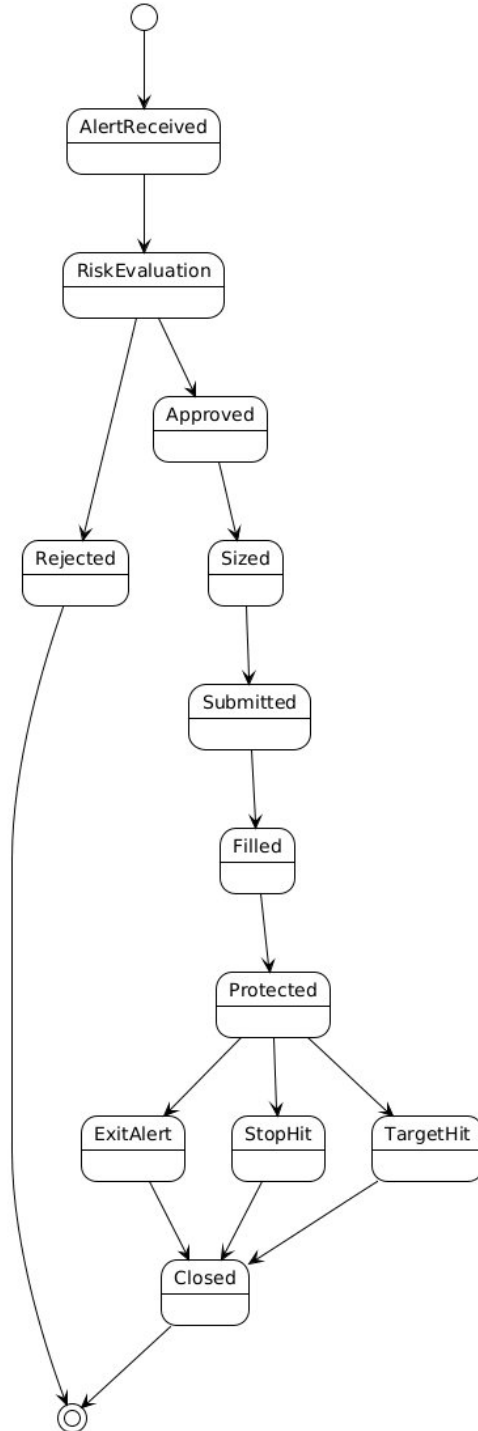
5.3 Data Flow

Both ingestion paths converge on one processing path: an alert is qualified, normalized, classified, and evaluated. Approved entries are sized, admitted by the guard, and submitted to the broker. Exits are matched against open positions and closed. Every processed alert, whether approved or rejected, is persisted along with its risk decision, and every execution produces a recorded outcome and a notification. Deduplication prevents the two ingestion paths from acting on the same alert, and a uniqueness constraint at the data store is the final guarantee against concurrent insertion.

6. Process View

A trade moves through a defined sequence of states from the moment an alert is received to the point the position is closed. An alert is risk-evaluated and either rejected or approved; an approved alert is sized, submitted, filled, and protected; and a protected position is closed by an exit alert, a stop, or a target. The state model below frames the per-process workflows described in the following subsections.

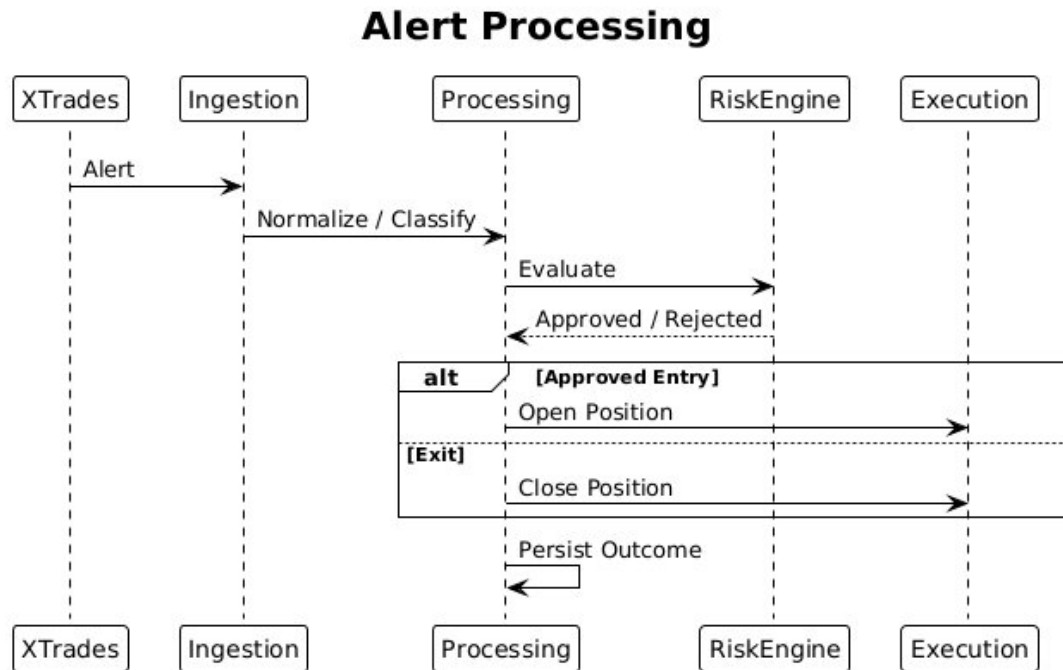
Trade Lifecycle



6.1 Alert Processing

Ingestion is split across two long-running services that share one processing path. The polling service provides a reliable source that continues to function when the real-time feed is degraded. The real-time service minimizes latency by reacting to alerts as they are published.

The real-time path decouples receipt from processing so that a burst of alerts cannot stall the connection, favouring connection stability and the freshest alerts when the system is briefly overwhelmed. Each path isolates its own failures: a failed cycle or a dropped connection is recovered without affecting the rest of the system.

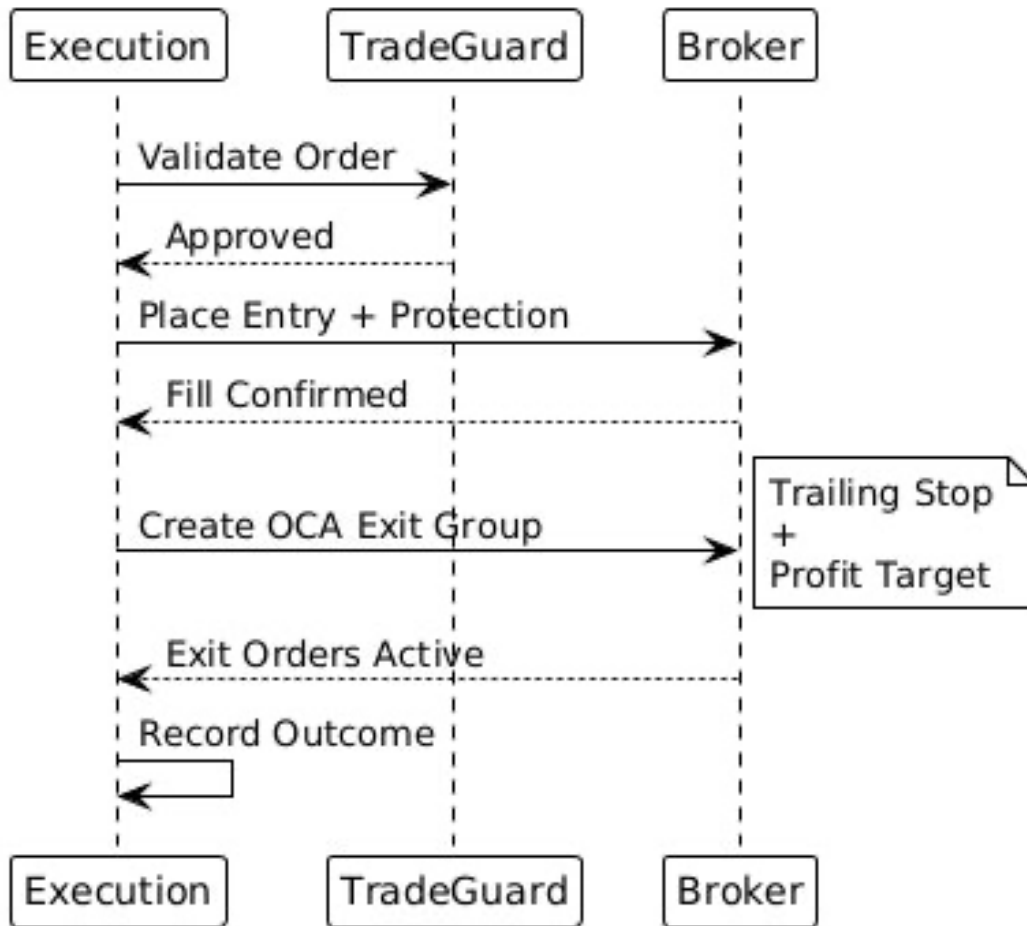


6.2 Trade Execution

Execution runs through a single path shared by both ingestion services. An approved entry is sized into an order, admitted by the trade guard, and placed with the broker. Entries are suppressed when the market is closed or when execution has been paused for the session.

The exit protection follows a deliberate sequence dictated by broker behaviour. A position is opened with immediate stop protection in place, and once the fill is confirmed, the protective orders are replaced with a trailing stop and a profit target held together in an OCA group, so that either exit cancels the other. Short-dated options omit the fixed target and run under the trailing stop alone, which avoids margin rejections and allows a winning short-dated position to run. The execution path tolerates a late fill confirmation from the broker so that a slow acknowledgement is not mistaken for a failed order.

Trade Execution



The exit path matches a close alert to a held position and instructs the broker to cancel the resting orders and close at market.

6.3 Position Monitoring

Exit orders rest with the broker, so the broker, not the system, detects when a stop or target is reached. The position monitor reacts to broker-side fills and reconciles the corresponding position, which keeps the system's view consistent without continuous price polling. Because exit orders also outlive a process restart, the monitor re-establishes its awareness of resting orders when state is reloaded, so a broker-side exit that occurs after a restart is still recognized.

6.4 Background Processes

A scheduler runs time-of-day tasks on market days only, skipping weekends and recognized holidays and evaluating all timing in Eastern Time. Its responsibilities are to establish the day's market regime at the open, to report operational health and position summaries, and to perform end-of-day handling.

The market regime is established once each morning from market-wide indicators and expressed as a chop score. A regime judged choppy automatically suppresses higher-risk activity for the session and can tighten the day's loss limit, independent of the standing configuration. This gives the system a daily, condition-aware posture without operator involvement. Performance reporting runs as a separate process against the recorded trade outcomes, isolating analytical workloads from the trading runtime.

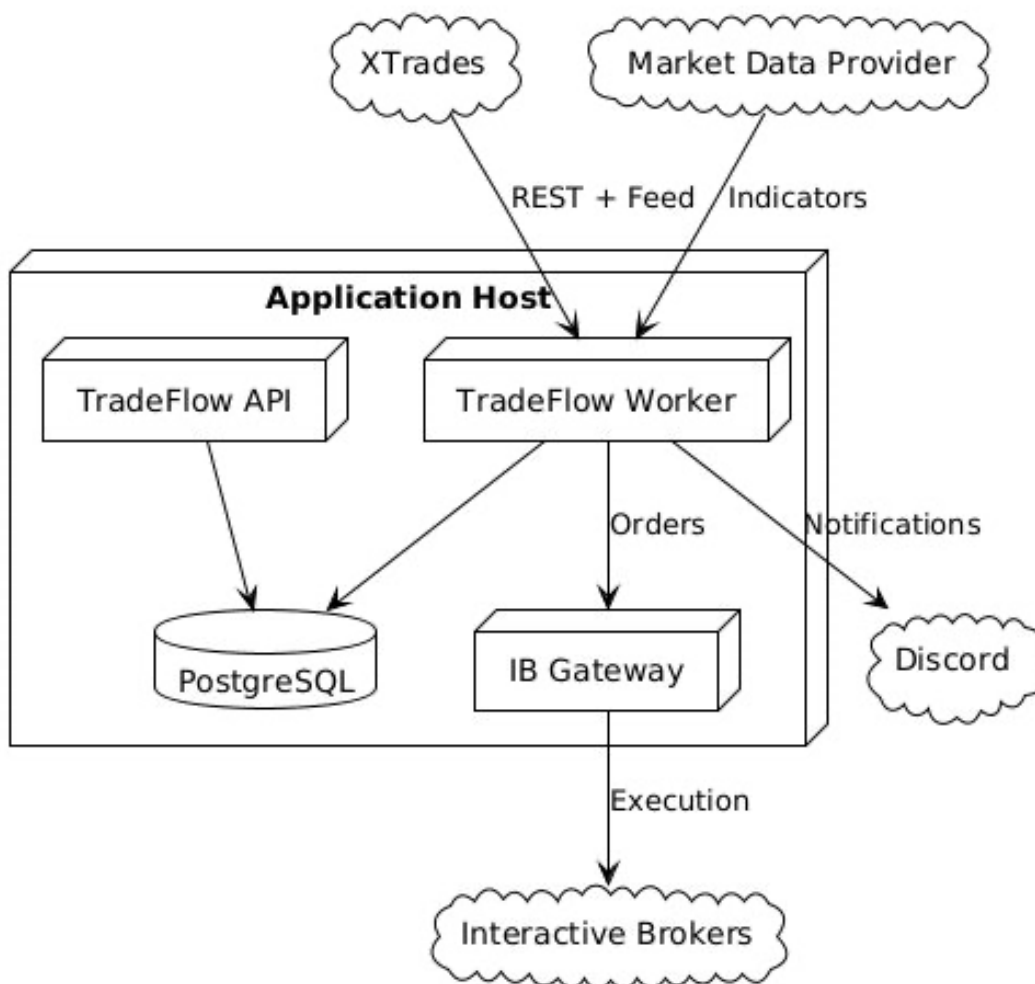
7. Deployment View

7.1 Runtime Environment

The execution core and the read interface are packaged as container images that run on the .NET runtime as a non-root user, with credentials supplied at runtime. At the time of writing the system runs against an Interactive Brokers paper account, with IB Gateway co-located with the execution core on the same host. The broker connection defaults to the paper endpoint and can be directed at the live endpoint through configuration.

7.2 Deployment Topology

TradeFlow - Deployment Architecture



7.3 Infrastructure Components

The data store is PostgreSQL, accessed through Entity Framework Core. It holds three stores: ingested alerts with their risk decisions, recorded trade outcomes for analysis, and open positions as the durable record of live exposure. The alerts and outcome stores are indexed to support the read interface and reporting queries. Open positions are keyed by the broker order that opened them, which is what allows the system to reconcile broker-side fills after a restart.

7.4 External Integrations

TradeFlow integrates with four external systems. Xtrades is the alert source, reached over both a polling interface and a real-time feed under a single credential. Interactive Brokers is the execution venue, reached through IB Gateway over a single serialized session. Discord receives operational and trading notifications across separated channels. A public market-data source supplies the indicators used to establish the daily regime.

8. Security Architecture

Secrets are externalized. Credentials and endpoints, including the Xtrades token, the notification webhooks, the database password, and the broker account identifier, are supplied at runtime, and the example configuration committed to the repository contains keys without values. The execution core refuses to start when essential configuration is absent, which prevents operation in a partially configured state. Deployable images run as a non-root user and are built so that credentials are never embedded in them.

The read interface exposes retrieval only. It enforces transport security outside test environments, returns structured error responses without internal detail in non-development environments, validates request parameters, and exposes its interactive explorer only in development.

The principal financial control is layered. The risk engine acts as an admission filter that rejects ineligible alerts before any sizing occurs. The trade guard acts as the final account-level control before capital is committed, preventing duplicated exposure on a single underlying, enforcing position and exposure limits, refusing to deploy more than available capital, and halting new entries after a daily loss limit is reached. The daily market regime can tighten these controls automatically in unfavourable conditions. Finally, the choice between simulation and live execution defaults to simulation, so a misconfigured environment cannot place real orders.

9. Performance and Scalability

TradeFlow is sized for a single operator and a single broker session rather than for horizontal scale, and its design reflects that intent.

The read interface caches results for a short interval and serves filtered, paginated queries against indexed data. Ingestion deduplicates alerts in bulk rather than per alert, keeping data-store interaction proportional to volume. On the execution path the account-level safety checks operate against cached account state refreshed in the background, so admission does not incur a broker round trip per order. The real-time ingestion path absorbs bursts without blocking receipt, trading the oldest unprocessed alerts for connection stability when briefly overwhelmed.

Order throughput is bounded by the single broker session, which is a constraint of the broker API rather than of the application. The architecture does not attempt to exceed it, and the system's expected alert volume operates well within it.

10. Reliability and Recovery

The system is designed to run unattended and to recover from the failure modes it encounters in operation.

Open positions are persisted and treated as the source of truth, so a restart does not lose awareness of live exposure. On recovery the system rebuilds its in-memory position state and re-establishes its awareness of the resting exit orders held with the broker, so a stop or target that fills during or after a restart is still recognized and reconciled.

The broker connection is hardened against the faults specific to a long-lived market session. Malformed market data does not terminate the session's message handling, the session is kept alive through quiet periods, and connection loss is detected promptly, surfaced as a critical notification, and followed by automatic reconnection. Order identifier management is reconciled on startup against both the broker and the recorded history to avoid the collisions that arise from the broker's periodic counter reset. Certain broker status and error codes that are informational in normal operation are recognized as such rather than treated as failures.

Each runtime workflow isolates its own failures. A failed ingestion cycle is retried on the next interval, the real-time connection reconnects on its own, and notification failures never block trade processing. A position recovery service that reconciles persisted positions against current broker state after an outage is part of the planned work and is recorded under Future Considerations.

11. Technology Stack

Layer	Technology
Runtime	.NET 10
Hosting	Long-running host with background services; minimal HTTP API
Persistence	PostgreSQL via Entity Framework Core
Broker	Interactive Brokers via IB Gateway (TWS socket API)
Real-time feed	Xtrades SignalR notification hub
Configuration	Strongly typed options validated at startup
Logging	Serilog with console and file outputs
Notifications	Discord webhooks across separated channels
Market data	Public market-data source for index and volatility indicators
API surface	Minimal API with output caching and health checks
Testing	xUnit with mocking, a containerized database, and an in-process API host
Packaging	Container images on the .NET runtime, running as a non-root user

12. Architectural Decisions

The following records the significant decisions evident in the system, each with its context, rationale, and tradeoffs.

ADR-1: Two ingestion paths feeding one processing path

Decision. Receive alerts through both a polling interface and a real-time feed, and route both into a single processing path.

Context. A real-time feed minimizes latency but can degrade or disconnect. Polling is dependable but introduces delay. Execution quality depends on acting quickly, while correctness depends on not missing alerts.

Rationale. Running both paths obtains low latency from the feed and reliability from polling, while a single shared processing path guarantees identical behaviour regardless of source.

Tradeoffs. The same alert can arrive on both paths, which requires deduplication and a uniqueness guarantee at the data store. The two paths must be kept behaviourally aligned as the system evolves.

ADR-2: Broker access behind a port with a startup-selected implementation

Decision. Depend on a broker abstraction and select the simulation or live implementation at startup, defaulting to simulation.

Context. The system must be testable without a broker connection, must support paper trading, and must eventually trade live, without divergent execution code.

Rationale. One execution workflow exercised in all three modes reduces the risk that live behaviour differs from tested behaviour. Defaulting to simulation prevents an unconfigured environment from placing real orders.

Tradeoffs. The abstraction must express the union of what the implementations support, and the simulation implementation must remain faithful enough to be a useful proxy.

ADR-3: OCA exit group rather than a fixed bracket

Decision. Protect a position with a trailing stop and a profit target held in a One-Cancels-All group, established after the entry fill is confirmed.

Context. The broker rejects a trailing stop attached as a bracket child, yet a position must be protected from the moment it opens.

Rationale. Opening with immediate stop protection and then replacing it with a trailing stop and target in an OCA group satisfies both constraints: protection is continuous, and the trailing exit is expressed in a form the broker accepts.

Tradeoffs. The two-step placement is more involved than a single bracket and depends on a confirmed fill before the final exit structure exists. Short-dated options run without a fixed target to avoid margin rejection, accepting an unbounded upside exit in exchange for the position remaining open.

ADR-4: Persistence as the source of truth for open positions

Decision. Persist open positions and rebuild in-memory state and broker-side awareness from them on startup.

Context. The system runs unattended for long periods and will be restarted. In-memory-only state would be lost, and exits could be missed.

Rationale. A durable record of live exposure allows the system to recover its position awareness and re-establish its awareness of resting exit orders after a restart.

Tradeoffs. Persisted state must be kept consistent with the broker, and recovery must reconcile against actual broker state to avoid acting on a stale record.

ADR-5: Risk policy as an ordered admission gate

Decision. Evaluate every alert against an ordered set of rules assembled from configuration, and persist both approvals and rejections.

Context. Eligibility depends on multiple independent conditions, the policy changes more often than the code, and decisions must be auditable.

Rationale. An ordered rule set returning the first failure makes the policy explicit and inexpensive to evaluate, configuration-driven assembly allows policy change without code change, and retaining rejected alerts preserves an audit trail.

Tradeoffs. Rule ordering carries meaning that must be maintained, and a configuration-driven policy shifts part of the system's behaviour into operational control.

ADR-6: A single serialized broker session

Decision. Route all broker interaction through one connection.

Context. The broker client is single-threaded by contract.

Rationale. Serializing access through one connection respects that contract and avoids the faults that concurrent access would cause.

Tradeoffs. Order throughput is bounded by a single session. This is acceptable given the system's volume but precludes scaling execution by adding sessions without further design.

ADR-7: Daily market regime as an automatic posture control

Decision. Establish a market regime once each morning and let it override risk posture for the session.

Context. Conditions that warrant a more conservative posture can be assessed at the open, and an unattended system cannot rely on an operator to adjust posture intraday.

Rationale. A daily regime lets the system suppress higher-risk activity and tighten loss limits automatically in unfavourable conditions.

Tradeoffs. The regime is fixed for the session and does not adapt to conditions that change after the open, and it depends on an external market-data source being available each morning.

13. Future Considerations

The following capabilities are planned but not yet implemented. They are recorded so the boundary between current and intended functionality is explicit.

Position recovery service. A service that, after an outage, reconciles persisted positions against current broker state and detects exit signals missed while the system was offline. Supporting configuration for the behaviours such a service would coordinate already exists.

Deployment automation. Automated build and deployment of the execution core and read interface, and automated recovery of the broker session, so that routine operation requires no manual steps.

Operational hardening. Continued strengthening of unattended operation, including health monitoring and alerting around the broker session and the ingestion paths.

Architectural improvements. Alignment of the notification component with the shared outbound HTTP infrastructure used elsewhere in the execution core, and end-to-end confirmation that risk and sizing controls present in configuration are exercised by an active code path before they are relied upon in live trading.

This document should be revised as these capabilities are delivered so that the recorded architecture continues to match the system as built.